DEEPSIGHT

Revised Master Build Specification | GenLayer Proof of Discovery

Architecture decision record + implementation addendum | 13 October 2026

This revised edition supersedes earlier suggestions to use fictional GenLayer verification hooks or immediate GEN bounties. It includes the original full technical blueprint as an annex, with this addendum taking precedence wherever details differ.

Executive build decision

Build DeepSight as a scientific analysis platform first. Use a single GenLayer-powered Proof of Discovery (PoD) protocol with two policy modules: Celestial Object Evidence Verification and Radio Signal Integrity. These evaluate evidence sufficiency and provenance—not whether a newly discovered planet or extraterrestrial intelligence has been scientifically confirmed.

• Phase 1: CPU-first TESS light-curve ingestion, BLS transit analysis, catalogue checks, evidence manifests and on-chain claim lifecycle.

• Phase 2: celestial-object evidence policy, human reviewer queue, disputes and independent scientific attestations.

• Phase 3: radio signal analysis and Signal Integrity policy, then optional carefully governed incentives.

• Keep all raw scientific computation off-chain; submit immutable hashes, concise measurements and policy-specific claims to GenLayer.

01 / Updated architecture and responsibilities

```
Next.js / TypeScript scientific workspace
  |-- Explorer / Discovery Lab / Evidence / Claims / Notebook
  v
FastAPI + Postgres + Redis + object storage
  |-- MAST/TESS + NASA Exoplanet Archive + Gaia + JPL
  |-- CPU analysis: Astropy, Lightkurve, SciPy, photutils
  |-- Radio analysis: staged, optional V2
  |-- Evidence manifest builder + source snapshot service
  v
GenLayer adapter (genlayer-py / genlayer-js)
  |-- ProofOfDiscoveryRegistry (claim lifecycle)
  |-- CelestialEvidencePolicy (evidence review)
  |-- SignalIntegrityPolicy (radio evidence review)
  |-- Appeals / attestations (network-supported workflow)
  v
Finalized claim history + off-chain indexed read model
```

02 / What GenLayer actually verifies

Evaluation dimension	Allowed outcome
Provenance	Source references and archive snapshots match declared inputs.
Reproducibility	Manifest includes code revision, parameters, checksums and analysis outputs.
Catalogue matching	The declared comparison to named catalogues is adequately documented; lack of a match does not prove novelty.
Scientific measurements	Declared calculations and units are consistent with attached analysis, subject to bounded verification.
Final protocol result	EVIDENCE_SUFFICIENT / INSUFFICIENT / NEEDS_HUMAN_REVIEW. Not “planet confirmed” or “alien signal verified.”

03 / Two verification modules

A. Celestial Object Evidence Verification

Input: target identifier, coordinates and reference frame, observation IDs, FITS/light-curve checksums, extracted metrics with uncertainties, cross-match query snapshots, plots and methodology. Evaluation: required artifacts, source integrity, duplicate claim check, reasonable method disclosure, documented false-positive tests. Output: protocol evidence decision plus reviewer flags. Scientific discovery status is managed separately and cannot be set to “confirmed” by this contract.

B. Radio Signal Integrity

Input: observation ID, frequency range, time range, bandwidth, drift rate, signal-to-noise method, instrument calibration, waterfall plots, raw-data reference, RFI exclusion tests and off-target comparisons where available. Evaluation: evidence integrity, methodology completeness and presence of required interference controls. Output: anomaly-evidence decision, never an extraterrestrial-origin determination.

Shared claim lifecycle

DRAFT → EVIDENCE_READY → SUBMITTED → PENDING_CONSENSUS → PROVISIONAL_DECISION → FINALIZED; with NEEDS_REVIEW, CHALLENGED, SUPERSEDED and RETRACTED tracked explicitly. Translate these product states to actual GenLayer transaction and appeal states; do not invent native network statuses.

04 / Contract implementation rules

• Use the official GenLayer Intelligent Contract boilerplate and its currently documented Python contract syntax. Do not copy standard Python classes and assume they are deployable contracts.

• Do not call genlayer.verifycelestial, genlayer.verifysignal, genlayer.lock_bond or genlayer.distribute: these were speculative examples, not verified APIs.

• Use GenLayer validator consensus and its supported nondeterministic/web-data primitives only as documented by the installed runtime version.

• Prefer one registry plus policy version identifiers over per-claim deployments. Add separate contracts only if isolation or upgrade requirements justify them.

• No staking, slashing, GEN rewards, marketplace or bond escrow in MVP. Any later financial contract needs a funded treasury, payout semantics, attack analysis and audited implementation.

• Do not use an IPFS CID as a wallet address; do not infer that an IPFS CID means a file is permanently pinned.

Contract-facing domain interface (design contract, not deployable SDK code)

```
register_claim(manifest_hash, claim_type, policy_version, source_ref)
read_claim(claim_id)
request_evidence_review(claim_id, evidence_summary_hash)
record_policy_outcome(claim_id, enum_result, rationale_hash)
supersede_claim(claim_id, new_manifest_hash)
# challenge/appeal actions depend on actual GenLayer network support
```

The functions above are proposed application interfaces. Implement them using actual GenLayer contract decorators, transaction handling, client methods and consensus primitives from official documentation; do not assume the names already exist.

05 / Canonical evidence requirements

```
{
  "schema_version": "1.1",
  "claim_type": "celestial_object_evidence",
  "policy_version": "celestial-v1",
  "target": {"id": "TIC-EXAMPLE", "ra_deg": 0, "dec_deg": 0},
  "archive": {"name": "MAST", "observation_ids": ["..."]},
  "analysis": {"git_commit": "...", "container_digest": "..."},
  "metrics": [{"name": "period", "value": 2.5, "unit": "d"}],
  "artifacts": [{"sha256": "...", "uri": "s3://..."}],
  "cross_matches": [{"catalogue": "NASA Exoplanet Archive",
                     "query_snapshot_sha256": "..."}],
  "scientific_status": "unconfirmed_candidate"
}
```

• Canonicalize and SHA-256 hash exact manifest bytes; preserve object checksums, schema version and policy version.

• Use signed URLs for private evidence; do not expect validators to download multi-GB datasets. Publish small independently checkable summaries or snapshots.

• Use content-addressed IPFS as an optional distribution layer, not the only durable store; configure pinning, backups and retention.

• Add uncertainty and limitations fields for each claim; distinguish an archive “no match” from novelty proof.

06 / API and SDK integration plan

Subsystem	Implementation
Scientific ingestion	astroquery.mast, astroquery.gaia, NASA Exoplanet Archive TAP/ADQL, pyvo; JPL Horizons where applicable.
Analysis	Astropy, Lightkurve, NumPy, SciPy, pandas; optional photutils and scikit-learn.
Backend	FastAPI, Pydantic, SQLAlchemy, Alembic, Redis and background worker.
GenLayer	genlayer-py server-side; genlayer-js for wallet/client flows; official boilerplate and GLSim/Studio testing.
MCP	Official MCP Python SDK: astronomy-mcp, analysis-mcp, evidence-mcp, genlayer-mcp; read-only defaults.
Frontend	Next.js, Tailwind, shadcn/ui, Plotly.js, Aladin Lite; wallet signing gated by explicit user action.

Required additional endpoints

```
POST /v1/evidence/preflight
POST /v1/claims/{id}/review-request
GET  /v1/claims/{id}/policy-result
GET  /v1/claims/{id}/finality
POST /v1/claims/{id}/supersede
GET  /v1/research/attestations?claim_id=...
# Later: /v1/radio/observations and /v1/radio/analyses
```

07 / Professional frontend: final design direction

Visual abstract: a precision astronomical research workstation, not a generic Web3 dashboard. Use near-black/navy observation mode, clean off-white paper mode, spectral teal for primary actions, thin dividers, measured typography and real plots. Avoid gradients, glass effects, random planet renders and fake discovery statistics.

• Landing: compact navigation, “Explore the unknown. Verify the evidence.” hero, real light-curve visualization, three-stage Discover → Analyze → Verify, archive credits and clear research limitations.

• Discovery Lab: left target/observation sidebar, central light curve + folded transit plot, right metrics/quality flags; analysis parameters and source provenance always visible.

• Evidence Builder: scientific checklist, source citations, artifact checksums, reproducibility scorecard and human sign-off; disabled submission with explicit missing-item reasons.

• Claim detail: separate “Scientific status: unconfirmed” from “Protocol evidence: sufficient”; show transaction status, policy version, reviewer flags, challenge state and hash.

• Radio Signal Integrity (V2): frequency-time waterfall plot, RFI exclusion controls, drift-rate evidence and warning that anomalies are not technosignatures.

• Responsive: desktop-first analysis, mobile-friendly browsing; keyboard accessibility, reduced motion, units and downloadable figure alternatives.

08 / Roadmap, exit criteria and hard stops

Milestone	Release gate
M0 / Foundation	Source adapters, auth, job queue, object storage, dashboard shell.
M1 / Real science	TESS light curve processed on CPU with reproducible BLS result and known false-positive tests.
M2 / Proof of Discovery	Canonical manifest, on-chain registry, fees, wallet signing, finalized transaction and indexed read model.
M3 / Celestial policy	Review criteria, GenLayer integration tests, human-review fallback and public claim detail.
M4 / Public beta	Security, archive attribution, reliability, usability and methodology documentation.
M5 / Radio (later)	Sampled public radio data, RFI tests, signal integrity policy and specialist review.

Do not ship rewards until dispute/finality behavior, treasury custody, exploit resistance and scientific governance have been independently reviewed. Do not advertise “AI-proven discoveries.”

09 / Source and documentation checklist

Official references to validate during implementation: GenLayer docs https://docs.genlayer.com/ ; GenLayer SDK references https://docs.genlayer.com/api-references/genlayer-py and https://docs.genlayer.com/api-references/genlayer-js ; NASA Exoplanet Archive https://exoplanetarchive.ipac.caltech.edu/ ; MAST https://mast.stsci.edu/ ; Gaia https://gea.esac.esa.int/archive/ ; JPL https://ssd-api.jpl.nasa.gov/doc/horizons.html ; Astropy https://docs.astropy.org/ ; Lightkurve https://docs.lightkurve.org/ ; MCP https://modelcontextprotocol.io/. Check current methods, network deployment requirements, permissions and fees before coding.

The following annex contains the broader original specification: detailed system components, datasets, database schema, MCP tools, REST APIs, scientific pipelines, test strategy, operations and frontend wireframe. Where it differs, the revised decisions above control.

DEEPSIGHT

GenLayer-Powered Astronomical Discovery & Scientific Evidence Protocol

MASTER BUILD SPECIFICATION | Version 1.0 | 13 October 2026

Implementation blueprint • system architecture • astronomy APIs • SDKs • MCP servers • intelligent contracts • scientific workflows • professional frontend design • testing • launch plan

Product thesis

DeepSight is a research-grade, AI-assisted platform for searching public astronomical archives, executing reproducible analysis, ranking unusual signals and celestial-object candidates, and submitting evidence-backed claims to a GenLayer Intelligent Contract for protocol-level adjudication. GenLayer does not perform the astronomy and does not certify scientific discovery: it adjudicates explicitly defined evidence requirements and maintains an auditable claim history.

Build decision: start with one scientifically credible end-to-end workflow (TESS exoplanet transit candidate analysis), then add Gaia cross-matching, MAST image anomalies, radio data and collaborative research. Ship CPU-first, with GPU acceleration optional.

01 / Outcomes, users and product boundaries

- Primary users: astronomers, students, citizen scientists, observatory teams, scientific reviewers and protocol contributors.
- Primary outcome: a researcher can select a dataset, execute an analysis, inspect figures and metrics, produce a reproducible evidence bundle, and submit a claim for GenLayer adjudication.
- Secondary outcomes: catalogue cross-matching, traceable research notebooks, transparent challenge history, shared workspaces and publicly browseable findings.
- Not a promise: detecting aliens, commanding NASA telescopes, accessing live Voyager telemetry, or conclusively identifying new planets without independent observation.
- Core distinction: protocol-verified evidence compliance is not peer review, scientific confirmation, or IAU designation.

02 / Scope and release strategy

Release	Included	Excluded
MVP (6-10 weeks)	TESS/MAST import, light curves, transit search, archive cross-match, evidence bundles, GenLayer claim registry, dashboard	Radio signal discovery, custom telescope control, payouts
V1 (next 6-10 weeks)	Gaia/Exoplanet Archive, image anomaly workflow, reviewer queues, challenges, research notebooks, public explorer	Massive raw-data ingestion
V2	Breakthrough Listen sampled signal workflow, organization collaboration, external researcher attestations, optional bounties	Automated scientific certification

03 / Reference architecture

```
WEB (Next.js / React / TypeScript)
  |-- Explore / Discover / Analyze / Claims / Research / Settings
  |-- Authentication + GenLayer wallet + status timeline
  v
API GATEWAY (FastAPI) ---> PostgreSQL + pgvector
  |                         Redis queue / cache
  |--> Ingestion adapters --> NASA MAST / Exoplanet TAP / Gaia / JPL
  |--> Analysis workers --> Astropy / Lightkurve / SciPy / BLS
  |--> AI research agent --> tools + grounded retrieval + MCP
  |--> Evidence builder --> S3-compatible object storage
  |--> GenLayer adapter --> GenLayerJS / GenLayerPY
                                  |
                            Intelligent Contracts
                            Registry + Review + Appeals
```

Keep astronomy computations and raw observations off-chain. Store immutable manifests, hashes, dataset identifiers, evaluation-policy versions, timestamps, claimant identities and decision history on-chain or in contract-linked records. Use a read model indexed from finalized GenLayer events/transactions.

04 / Technology stack and prerequisites

Layer	Preferred stack	Reason
Frontend	Next.js, TypeScript, Tailwind CSS, shadcn/ui, TanStack Query, Plotly.js, Aladin Lite	Fast accessible scientific UI, interactive plots and sky visualization
API	Python 3.12+, FastAPI, Pydantic v2, SQLAlchemy, Alembic	Strong science ecosystem
Workers	Celery or Dramatiq + Redis; Docker	Async CPU jobs and retries
Science	Astropy, Astroquery, Lightkurve, NumPy, SciPy, pandas, photutils, scikit-learn	Astronomy-specific reproducible analysis
Data	PostgreSQL + pgvector; S3-compatible storage; Redis	Metadata, retrieval, large evidence artifacts
Contracts	GenLayer Python Intelligent Contracts, GenLayerJS, genlayer-py, genlayer-test	Judgment-based claim evaluation
Infrastructure	Docker Compose initially; VPS + object storage; CI/CD	Low-cost deployment and portable services

- Development machine: 16 GB RAM recommended; 8 GB works for small datasets. CPU-only supported for MVP. Node.js 18+, Python 3.12+, Git, Docker for local full Studio.
- Secrets: external API tokens where required, LLM key, DB URL, storage credentials, GenLayer RPC and signing credentials. Never put private keys or archive tokens in browser bundles.
- Use dependency lockfiles, container images pinned by digest, database backups and structured logs.

05 / Astronomical data integrations: verified source map

Provider	Integration	Initial use
NASA Exoplanet Archive	TAP/ADQL; tables ps, pscomppars, toi	Known planets, candidates, parameters and cross-check
STScI MAST	MAST REST API or astroquery.mast	TESS/Kepler observation search, light-curve downloads
ESA Gaia	Gaia Archive TAP/ADQL through astroquery.gaia or pyvo	Astrometry, stellar matches, nearby sources
NASA JPL Horizons	JPL Horizons HTTP API / astroquery.jplhorizons	Ephemerides and solar-system object positions
ESO Science Archive	IVOA TAP / ADQL / pyvo	Optional imaging and spectra expansion
Breakthrough Listen	Berkeley SETI open data archive; staged file retrieval	Later-stage radio signal pipeline; multi-GB files

API caveats: public availability does not imply unlimited usage. Respect archive attribution, data rights, request limits, caching, bulk-download rules and versioned schemas. JPL SSD services require server-side access for websites; do not assume browser CORS support. Archive access is not direct control of telescopes or spacecraft.

Reference integration patterns

```
# Python packages (pin versions in lockfile)
pip install fastapi uvicorn astroquery astropy lightkurve \
    numpy scipy pandas scikit-learn photutils pyvo \
    sqlalchemy alembic psycopg[binary] redis celery

# Example Exoplanet Archive TAP endpoint:
# https://exoplanetarchive.ipac.caltech.edu/TAP/sync
# GET ?query=select+top+10+pl_name+from+ps&format=json
# Validate actual table schema and query encoding at runtime.
```

06 / Discovery and scientific analysis pipelines

A. TESS transit candidate workflow (MVP)

- User selects a target identifier or sky coordinates; MAST adapter resolves available TESS sectors and observation products.
- Ingest the smallest relevant light-curve product; persist source DOI/URL, observation ID, checksum, acquisition time and archive response metadata.
- Clean flagged points, normalize flux, assess gaps and systematics, preserve original and processed versions.
- Run Box Least Squares period search (Astropy/Lightkurve), generate periodogram and phase-folded curve.
- Compute signal-to-noise proxy, transit depth, estimated period/duration, odd-even transit comparison, secondary eclipse check and data-quality flags.

- Cross-match against NASA Exoplanet Archive TOI/known planets and Gaia stellar neighbors; flag possible blends and known candidates.
- Produce a candidate report with plots, code version, parameters, uncertainty, caveats and reproducibility script.
- Require human review before making a public scientific claim; route an evidence-completeness claim to GenLayer.

B. Gaia catalogue anomaly workflow (V1)

- Run constrained ADQL cone searches; cache query, archive version and coordinate frame.
- Identify outliers using physically meaningful features and uncertainty-aware thresholds; do not equate statistical outlier with new object.
- Cross-match SIMBAD/VizieR where permitted and annotate known sources, motion and possible artifacts.

C. Image analysis workflow (V1)

- Download selected FITS image cutouts, validate WCS and calibration provenance, extract sources with photutils.
- Compare aligned epochs where available; detect candidate transients; mask cosmic rays, saturated pixels, artifacts and edge effects.
- Rank for human follow-up, preserve difference images and photometric measurements.

D. Radio analysis workflow (V2)

- Use sampled publicly available Breakthrough Listen filterbank/spectrogram data; keep large files in object storage or process remotely.
- Generate waterfall plots; detect narrowband features, drift rates and RFI-like signatures; compare off-target observations where available.
- Label anomalies 'unexplained signal candidate', never 'extraterrestrial transmission' without extraordinary corroboration.

07 / GenLayer: exact responsibilities

- DiscoveryRegistry: claim IDs, submitter, manifest hash, dataset reference, protocol status, revision links and finalized decision records.
- EvidenceReview: validate permitted evidence schema, source provenance and explicit policy predicates; perform bounded LLM/web-assisted judgment only where necessary.
- ChallengeManager: accept reasoned objections and evidence revisions; record status and apply native GenLayer appeal/finality mechanisms as supported by selected network.
- ResearchAttestations (later): attach independent expert statements without presenting protocol consensus as expert consensus.
- No custom token or reward system in MVP; no validator promises beyond actual network behavior.

Suggested claim states

DRAFT -> ANALYZED -> READY -> SUBMITTED -> PENDING_CONSENSUS -> ACCEPTED_PENDING_FINALITY / REJECTED_PENDING_FINALITY / UNDETERMINED -> FINALIZED_ACCEPTED / FINALIZED_REJECTED; plus CHALLENGED, SUPERSEDED, RETRACTED and FAILED_TRANSACTION. Exact chain states must map to the selected GenLayer SDK and consensus version.

Evidence schema (canonical JSON; hash SHA-256)

```
{
  "schema_version": "1.0",
  "claim_type": "transit_candidate_evidence",
  "target": {"id": "TIC-EXAMPLE", "ra_deg": 0.0, "dec_deg": 0.0},
  "source": {"archive": "MAST", "observation_ids": ["..."]},
  "pipeline": {"git_commit": "...", "container_digest": "..."},
  "parameters": {"period_days": 2.5, "snr": 8.2},
  "artifacts": [{"uri": "s3://.../curve.fits", "sha256": "..."}],
  "cross_matches": [{"archive": "NASA Exoplanet Archive",
                     "result": "no_exact_match"}],
  "limitations": ["Not scientifically confirmed"],
  "created_at": "2026-10-13T12:00:00Z"
}
```

Canonicalize JSON (stable key ordering and numeric representation), hash the canonical bytes, and sign the manifest. Immutable evidence references should be content-addressed or backed by retention controls. Never submit an unrestricted external URL that could change after evaluation.

Evidence adjudication policy

- Check manifest schema, source allowlist, artifact checksums, pipeline metadata and mandatory plots.
- Confirm cited source identifiers and cross-match assertions are consistent with archived snapshots where feasible.
- Output one bounded enum: EVIDENCE_SUFFICIENT, EVIDENCE_INSUFFICIENT, NEEDS_HUMAN_REVIEW. Include short rationale and policy version.
- Equivalence rule: validators compare the enum and required objective criteria, not unconstrained prose similarity. If uncertainty is material, return NEEDS_HUMAN_REVIEW.
- Restrict prompt-injection from external metadata, cap external fetch sizes and avoid asking validators to independently rerun huge scientific jobs.

GenLayer SDK and development workflow

```
git clone https://github.com/genlayerlabs/genlayer-project-boilerplate.git
cd genlayer-project-boilerplate
pip install -r requirements.txt
npm install -g genlayer
# Fast simulator (no Docker):
pip install 'genlayer-test[sim]'
glsim --port 4000 --validators 5
# Lint and test contracts:
genvm-lint check contracts/discovery_registry.py
gltest tests/integration/ -v -s
# Full local Studio requires Docker:
genlayer init
genlayer up
# Frontend client:
npm install genlayer-js
# Python backend client:
pip install genlayer-py
```

Contract syntax, GenVM dependency pin, network aliases, transaction fees and lifecycle are version-sensitive. Use the current official boilerplate and test against GLSim, full Studio, then the chosen persistent testnet. The documentation currently describes Bradbury, Asimov, Studionet and a separate Studio-dev preview: never assume they are interchangeable.

GenLayer implementation sequence

- Start with deterministic registry contract: create_claim(manifest_hash, source_id, claim_type), get_claim, supersede_claim; owner checks and duplicate protection.

- Add evaluation contract with a compact policy payload, web retrieval only to allowlisted source endpoints and a narrow equivalence rule.
- Estimate write/deploy fees with SDK, submit transaction, persist hash, wait for finalization, then inspect execution result (finalized does not imply success).
- Index transaction and status updates into Postgres with idempotent workers; expose a user-readable consensus and appeal timeline.
- Test inconsistent validators, source outages, malformed evidence, poisoned descriptions, retries, replay, insufficient fees, and appeal edge cases.

08 / MCP architecture and AI research agents

MCP is an optional tool interoperability interface, not a replacement for REST or scientific libraries. Build internal MCP servers after core Python tools are stable. Use the official Model Context Protocol Python SDK or TypeScript SDK, and expose narrowly scoped, auditable tools.

MCP server	Tools (proposed)	Permissions
astronomy-mcp	search_mast, query_exoplanets, cone_search_gaia, get_horizons	Read-only, allowlisted
analysis-mcp	run_transit_search, inspect_lightcurve, compare_catalogues, create_plot	Job-scoped, quotas
evidence-mcp	build_manifest, verify_hashes, retrieve_artifact	Read plus controlled writes
genlayer-mcp	read_claim, estimate_review_fee, submit_claim, get_tx_status	Writes require explicit user approval
research-mcp	search_project_notes, summarize_evidence, draft_report	Workspace-scoped

- MCP tool results must include provenance, units, timestamps and errors; use JSON schemas for arguments and responses.
- No autonomous wallet signing or irreversible publication from an LLM without explicit confirmation.
- Agent should cite data sources, never fabricate detections, and separate measured quantities from speculative interpretation.
- Expose MCP via authenticated HTTP or local stdio as appropriate; do not publicly expose privileged analysis or wallet tools.

09 / Backend API contract (v1)

Method / path	Purpose
POST /v1/auth/session	Establish authenticated session
GET /v1/sources	List supported archives and availability
POST /v1/datasets/search	Search MAST/TAP catalogues
POST /v1/datasets/import	Queue import, return job ID
GET /v1/datasets/{id}	Dataset metadata and provenance
POST /v1/analyses	Start analysis using pipeline + config
GET /v1/analyses/{id}	Status, metrics, warnings, artifacts
GET /v1/analyses/{id}/plots	Signed URLs to plots
POST /v1/evidence	Build immutable evidence manifest
POST /v1/claims	Create draft claim
POST /v1/claims/{id}/submit	Request GenLayer submission
GET /v1/claims/{id}/timeline	Consensus, appeal, finality events
POST /v1/claims/{id}/challenge	Challenge request (policy gated)
GET /v1/discoveries	Search public findings
GET /v1/jobs/{id}	Job progress / error state
GET /v1/health	Health and integration diagnostics

API standards: OpenAPI 3.1, Pydantic validation, cursor pagination, idempotency keys for POST actions, UTC timestamps, request IDs, rate limits, structured error codes and signed artifact URLs. Use SSE/WebSocket for job and transaction progress. Authentication: OIDC/email login plus optional wallet linking; authorization: owner/reviewer/admin roles.

10 / Database and storage design

- users(id, email, display_name, role, created_at); wallet_links(user_id, chain, address, verified_at).
- workspaces(id, owner_id, visibility); memberships(workspace_id, user_id, role).
- data_sources(id, provider, source_uri, license, fetched_at, checksum, archive_version).
- datasets(id, source_id, target_id, ra_deg, dec_deg, observation_id, object_key, status).
- analysis_runs(id, dataset_id, pipeline_name, pipeline_version, config_json, status, started_at, completed_at, error_code).
- measurements(id, run_id, name, value, unit, uncertainty, method); artifacts(id, run_id, kind, object_key, sha256).
- claims(id, owner_id, manifest_hash, evidence_key, chain, contract_address, chain_claim_id, status).

- claim_events(id, claim_id, tx_hash, block_ref, status, payload_json, observed_at); challenges(id, claim_id, reason, evidence_hash, status).
- audit_logs(id, actor_id, action, resource, trace_id, created_at); agent_tool_calls(id, actor_id, tool, args_hash, outcome).

Indexes: dataset coordinates via suitable spatial indexing, run status/time, claim manifest hash unique, transaction hash unique, full-text search on public descriptions. Partition large job logs. Do not store raw FITS, filterbank or multi-GB data as database blobs.

11 / Professional frontend abstract and visual system

Design direction: 'Mission Control meets scientific research journal'. Premium, restrained, precise and research-first. Avoid gradients, glassmorphism, neon sci-fi clichés and oversized decorative planets. Favor disciplined typography, hairline separators, meaningful data density, deep midnight backgrounds in observation mode and a warm off-white research mode.

Token	Specification
Primary ink	Midnight #101B2A
Surface dark	#0B1420 / #132133
Surface light	#F7F9FA / white
Accent	Spectral teal #00A7A0; limited use
Warning / risk	Amber #C88C32; red #C95757
Typography	Inter / Geist for UI; IBM Plex Mono for coordinates, hashes, units
Layout	12-column grid, 8px spacing scale, 1px borders, 12px cards
Interactions	200ms transitions, accessible focus states, reduced motion support

Information architecture

- Public: Landing, Explore Universe, Public Discoveries, How Verification Works, Research Methodology, Documentation.
- Authenticated: Overview, Data Explorer, Discovery Lab, Analysis Runs, Evidence Builder, GenLayer Claims, Research Notebook, Team Workspace, Settings.
- Admin/reviewer: source health, job failures, abuse reports, review queues, contract configuration and audit events.

Landing page abstract

- Top bar: DeepSight wordmark, Explore, Discoveries, Research, Protocol, Docs; right: Sign in / Launch Workspace.
- Hero: 'Explore the unknown. Verify the evidence.' One short paragraph and two actions: Open Discovery Lab / Explore Findings.
- Hero visual: authentic scientific light-curve or celestial coordinate plot with provenance label; never a misleading AI-generated 'new planet' image.

- Proof strip: Supported archives, datasets analyzed, active research runs, protocol-reviewed claims; display only real metrics.
- Feature sequence: Discover / Analyze / Verify, each illustrated with a real product screenshot or data visualization.
- GenLayer explainer: three steps with status chips, evidence hash and a transaction timeline; explicit scientific caveat.
- Footer: methodology, source credits, reproducibility, policies, documentation.

Workspace layout: functional wireframe

```
+-------------------------------------------------------------------+
| DEEPSIGHT   [Search target / TIC / coordinates]  [Jobs] [Account] |
+-------------+-----------------------------------------------------+
| Overview    | Discovery Lab / TIC-...         [New Analysis]      |
| Explorer    |-----------------------------------------------------|
| Discover    | TARGET / OBSERVATION     | RESEARCH STATUS         |
| Analyses    | RA / DEC / Sector        | Data imported          |
| Evidence    |--------------------------+------------------------|
| Claims      | LIGHT CURVE / PERIOD SEARCH (interactive plot)      |
| Notebook    |-----------------------------------------------------|
| Team        | Metrics / Uncertainty / Quality Flags / Provenance |
| Settings    |-----------------------------------------------------|
|             | Evidence Checklist  |  Submit for GenLayer Review |
+-------------+-----------------------------------------------------+
```

Critical screens and acceptance criteria

- Explorer: search by name, coordinates, source, date; sky map and table synchronized; clear catalogue attribution.
- Discovery Lab: side-by-side raw/processed light curve, periodogram, folded curve, quality warnings and reproducible run settings.
- Evidence Builder: checklist, linked artifact hashes, sources, methodology, reviewer notes and preflight blockers.
- Claim Detail: on-chain reference, explicit evaluation scope, validator lifecycle, fee estimate, appeal window and finality badge.
- Research Notebook: markdown notes, code/config snapshots, pinned figures, citations, export to PDF/JSON.
- Every chart must label axes and units, include uncertainty/quality context, accessible text alternative and a download option.
- Mobile: browse findings and inspect claims; analysis workspace optimized for desktop/tablet, with graceful small-screen read mode.

12 / Scientific integrity, security and governance

- Scientific: document false-positive pathways, systematic noise, selection effects, uncertainty and reproducibility. Require human review for claims implying novel objects or extraterrestrial intelligence.
- Data: cite archive provenance and licenses; do not rebrand archive imagery as proprietary discoveries; record DOI where provided.
- Security: OWASP ASVS baseline, RBAC, session protections, CSP, strict CORS, SSRF-safe URL allowlists, malware scanning of uploads, signed object URLs.
- Contracts: bounded LLM/web calls, explicit source allowlists, prompt-injection defenses, authorization checks, replay protection, finality handling and independent code review.

- Privacy: minimize personal data, segregate public/private projects, redact secrets from evidence manifests and logs.
- Governance: policy versioning, claim retractions, appeals, reviewer conflict-of-interest disclosures and transparent correction records.

13 / Testing and quality gates

Layer	Required tests
Science	Known TESS signals, synthetic injections, negative controls, recovery precision/recall, numerical regression
API	Schema validation, authorization, pagination, idempotency, rate limits, timeout and archive outage
Workers	Retry semantics, cancellation, memory limits, duplicate jobs, artifact integrity
GenLayer	Direct-mode, GLSim, Studio integration, real testnet, fees, status, appeal, invalid evidence
Frontend	Playwright E2E, accessibility, responsive layouts, chart interactions, wallet errors
Operations	Backups/restore, storage lifecycle, logs/tracing, dependency scans, incident runbooks

MVP release gates: one documented real-data transit analysis from import to evidence manifest; repeatable result on pinned data; zero critical security issues; all on-chain status paths correctly represented; no scientific-confirmation claims in UI; accessible core flows; tested recovery from API and RPC outages.

14 / Execution roadmap and team work packages

Sprint	Build tasks	Exit criteria
0: Foundations	Repo, design system, auth, DB, storage, CI, source adapters	Health checks and source search working
1: Science ingestion	MAST import, metadata, job queue, provenance	Real TESS light curve imported
2: Discovery engine	BLS pipeline, charts, QA, archive cross-match	Reproducible candidate report
3: Evidence + GenLayer	Manifest hashing, registry contract, SDK, transaction lifecycle	Finalized claim visible in UI
4: Product polish	Lab, Explorer, Evidence, Claim detail, documentation	End-to-end demo + accessibility
5: Hardening	Tests, security, performance, monitoring, beta users	Release gates satisfied

Minimum delivery team: one full-stack engineer, one Python/scientific-computing engineer, one GenLayer contract engineer (roles may overlap), part-time astronomer or domain reviewer, and product designer. A solo builder should target a narrower 8-12 week MVP and reuse official boilerplates.

15 / Infrastructure, deployment and operating cost

- Local: Docker Compose for API, Postgres, Redis, object-store emulator, workers and frontend; GLSim separately for fast contract iteration.

- Production starter: one VPS for API/CPU workers, managed or self-hosted Postgres with backups, S3-compatible storage, frontend CDN, monitoring and a secure secrets manager.
- Use per-user job quotas and dataset-size ceilings; cache external catalogue queries; avoid bulk downloading raw radio archives.
- Measure monthly costs as VPS + storage capacity/egress + AI token usage + GenLayer transaction fees + monitoring. Do not quote fixed amounts before benchmarking.
- Upgrade path: separate science workers from API, managed queues, scheduled batch jobs, optional GPU worker pool for expensive inference only.

16 / Detailed build order and first commands

```
# Suggested monorepo
deepsight/
  apps/web/                 # Next.js UI
  services/api/             # FastAPI + routes
  services/worker/          # CPU analysis pipelines
  packages/schemas/         # OpenAPI / JSON schema
  packages/genlayer-client/ # wallet, fee, tx lifecycle
  contracts/                # Intelligent Contracts (Python)
  mcp/                      # Astronomy / analysis / evidence tools
  infra/                    # Compose, deployment, monitoring
  tests/                    # unit, scientific, e2e, contract
  docs/                     # ADRs, source policies, runbooks
```

- Day 1: initialize monorepo and build shell navigation, dashboard, empty-state screens and theme tokens.
- Day 2-3: connect MAST via Astroquery, import one TESS light curve, store FITS and metadata.
- Day 4-5: run BLS on CPU, plot light curve and fold, create signed evidence manifest.
- Day 6-7: scaffold GenLayer contract from official boilerplate, add claim create/read, run GLSim tests.
- Day 8-10: wire fee estimation, wallet confirmation, tx status and finalized result to claim detail.
- Then: harden, expand catalogue cross-checks, add reviewer challenge UI and public findings.

17 / Risks, non-goals and decision register

Risk	Mitigation
False scientific claims	Human review, confidence language, rigorous cross-checks, visible caveats
GenLayer version churn	Pin SDK/runtime; maintain network matrix; integration tests before upgrades
Slow archive endpoints	Caching, queued retries, circuit breakers, user-visible job states
High data/storage cost	Targeted cutouts, retention tiers, upload limits, sampled radio data
AI hallucination	Tool-grounded responses, evidence references, no autonomous scientific confirmation
Consensus ambiguity	Narrow policy outcomes, deterministic checks first, explicit equivalence tests
Scope explosion	Transit-first MVP; defer radio and broad marketplace

18 / Official references and implementation checklist

GenLayer developer setup — https://docs.genlayer.com/developers/intelligent-contracts/tooling-setup

GenLayer protocol — https://docs.genlayer.com/understand-genlayer-protocol

GenLayerJS — https://docs.genlayer.com/api-references/genlayer-js

GenLayerPY — https://docs.genlayer.com/api-references/genlayer-py

GenLayer deployment — https://docs.genlayer.com/developers/intelligent-contracts/deploying

NASA Exoplanet TAP — https://exoplanetarchive.ipac.caltech.edu/docs/TAP/usingTAP.html

MAST API — https://outerspace.stsci.edu/spaces/MASTDOCS/pages/113771434/Using+MAST+APIs

Gaia Archive — https://gea.esac.esa.int/archive/

JPL Horizons API — https://ssd-api.jpl.nasa.gov/doc/horizons.html

ESO programmatic access — https://archive.eso.org/programmatic/

Breakthrough Listen open data — https://seti.berkeley.edu/opendata

Astropy — https://docs.astropy.org/

Astroquery — https://astroquery.readthedocs.io/

Lightkurve — https://docs.lightkurve.org/

Model Context Protocol — https://modelcontextprotocol.io/

Implementation checklist: [ ] archive source policy [ ] source adapter tests [ ] transit pipeline [ ] evidence schema [ ] canonical hashing [ ] registry contract [ ] bounded review policy [ ] fee estimation [ ] finality tracking [ ] reviewer UX [ ] public methodology [ ] backups [ ] release tests.

Source review date: 13 October 2026. Links and API/SDK behavior should be checked against their official current documentation before pinning implementation versions. This is an engineering specification, not a claim that all listed features are currently implemented or that protocol-level review equals astronomical validation.